

死磕设计模式之如何抽象责任链模式

责任链模式

1. 什么是责任链

责任链设计模式是一种行为型设计模式，其主要目的是解耦请求发送者和请求接收者，让多个对象都有机会处理请求，从而避免请求发送者和接收者之间的紧耦合。

责任链模式的核心是一个链式结构，链中每个节点代表一个处理者对象，请求先经过第一个节点处理，如果该节点能够处理请求，则直接返回处理结果；否则，请求继续往下一个节点传递，直到找到能够处理该请求的节点为止。整个过程类似于流水线上的多个工作站，每个工作站负责一项工作，如果自己处理不了，就将工作交给下一个工作站，直到整个工作完成。

在责任链模式中，每个处理者对象都有一个指向下一个处理者对象的引用，这样就形成了一个处理者链。请求发送者只需要将请求发送给第一个节点即可，而不用关心请求会被哪个处理者对象处理。由于每个处理者对象都有机会处理请求，因此责任链模式可以实现请求的动态分配。

2. 优缺点

责任链模式的优点在于，它可以动态地添加、删除和调整处理者对象，从而灵活地构建处理链。同时，它也避免了请求发送者和接收者之间的紧耦合，增强了系统的灵活性和可扩展性。

不过，责任链模式也有一定的缺点，例如如果处理链过长或者处理时间过长，可能会对系统性能产生一定的影响。

3. 应用场景

在实际应用中，责任链模式常用于请求的预处理、请求的过滤、请求的分发等场景。例如，可以使用责任链模式来实现权限校验、日志记录、异常处理、请求重试等功能。同时，也可以将责任链模式与其他设计模式结合起来，例如装饰器模式、工厂模式、观察者模式等，从而实现更复杂的功能。

下单场景实战

1. 下单前置校验

在电商系统下单接口中，前置校验是非常重要的环节。下面是一个可能的校验步骤列表：

- 检查商品信息是否存在，包括商品名称、价格、规格等信息。
- 检查购买数量是否合法，是否超出了最大购买数量或最小购买数量的限制。
- 检查商品库存是否充足，以确保库存足够满足购买者的需求。
- 检查购买者的优惠券、积分等是否可以使用，以确保购买者能够享受相应的优惠或积分奖励。
- 检查收货地址信息是否完整和准确，以确保商品能够顺利地送达给购买者。
- 检查下单时间是否合法，例如检查购买者是否在限定的时间范围内下单。

真实电商场景中，验证逻辑绝对不仅仅是这些，过之而无不及。

对于完成这些前置校验逻辑，大部分程序员可能的代码思路如下：

```
public String createOrder(CreateOrderReqDTO xxx) {  
    // 检查商品信息是否存在，包括商品名称、价格、规格等信息  
    // 检查购买数量是否合法，是否超出了最大购买数量或最小购买数量的限制  
    // 检查商品库存是否充足，以确保库存足够满足购买者的需求  
    // 检查购买者的优惠券、积分等是否可以使用，以确保购买者能够享受相应的优惠或积分奖励  
    // 检查收货地址信息是否完整和准确，以确保商品能够顺利地送达给购买者  
    // 检查下单时间是否合法，例如检查购买者是否在限定的时间范围内下单  
    // .....  
}
```

解决前置校验需求需要实现一堆逻辑，常常需要写上几百上千行代码。

为了避免这种代码臃肿的情况，我们可以运用责任链设计模式，对下单验证逻辑进行抽象。

2. 责任链重构

定义一个责任链处理器接口，所有子任务都实现该接口以处理具体的业务逻辑。

同时，为了方便对责任链流程中的任务进行顺序处理，我们需要继承 Spring 框架中的排序接口 Ordered。这将有助于保证责任链中的任务顺序执行。

```

public interface OrderCreateChainHandler<T> extends Ordered {

    /**
     * 执行责任链逻辑
     *
     * @param requestParam 责任链执行入参
     */
    void handler(T requestParam);
}

```

创建一个责任链上下文容器，用于存储与责任链相应的子任务。

```

public final class OrderCreateChainContext<T> implements CommandLineRunner {

    private final List<OrderCreateChainHandler> orderCreateChainHandlerContainer = new
    ArrayList();

    /**
     * 责任链组件执行
     *
     * @param requestParam 请求参数
     */
    public void handler(T requestParam) {
        // 此处根据 Ordered 实际值进行排序处理
        orderCreateChainHandlerContainer.stream()
            .sorted(Comparator.comparing(Ordered::getOrder)).forEach(each ->
each.handler(requestParam));
    }

    @Override
    public void run(String... args) throws Exception {
        // 通过 Spring 上下文容器，获取所有 CreateOrderChainContext Bean
        Map<String, OrderCreateChainHandler> chainFilterMap =
        ApplicationContextHolder.getBeansOfType(OrderCreateChainHandler.class);
        // 将对应 Bean 放入责任链上下文容器中
        chainFilterMap.forEach((beanName, bean) ->
orderCreateChainHandlerContainer.add(bean));
    }
}

```

实现 `OrderCreateChainHandler` 接口作为责任链处理器，每个具体的实现类负责执行特定的逻辑。

```
// 订单创建参数必填检验
@Component
public final class OrderCreateParamNotNullChainHandler implements
OrderCreateChainHandler<OrderCreateCommand> {

    @Override
    public void handler(OrderCreateCommand requestParam) {
        // 逻辑执行
    }

    @Override
    public int getOrder() {
        return 0;
    }
}

// 订单创建参数正确性检验
@Component
public final class OrderCreateParamVerificationChainHandler implements
OrderCreateChainHandler<OrderCreateCommand> {

    @Override
    public void handler(OrderCreateCommand requestParam) {
        // 逻辑执行
    }

    @Override
    public int getOrder() {
        return 1;
    }
}

// 订单创建商品 SKU 库存验证
@Component
public final class OrderCreateProductSkuStockChainHandler implements
OrderCreateChainHandler<OrderCreateCommand> {

    @Override
```

```
public void handler(OrderCreateCommand requestParam) {  
    // 逻辑执行  
}  
  
@Override  
public int getOrder() {  
    return 2;  
}  
}
```

通过责任链模式优化，创建订单接口前置校验代码从上千行缩减为一行。

```
@RequiredArgsConstructor  
public class OrderServiceImpl implements OrderService {  
  
    private final OrderCreateChainContext<OrderCreateCommand> orderCreateChainContext;  
  
    public String createOrder(OrderCreateCommand requestParam) {  
        // 责任链模式：执行订单创建参数验证  
        orderCreateChainContext.handler(requestParam);  
    }  
}
```

经过责任链模式的重构，你是否发现业务逻辑变得更加清晰易懂了？采用这种设计模式后，增加或删除相关的业务逻辑变得非常方便，不再需要担心更改上千行代码的几行代码，导致整个业务逻辑受到影响的情况。

责任链抽象

可能细心的小伙伴会发现一个问题，当业务使用越来越多的情况下，重复定义 `OrderCreateChainHandler` 以及 `OrderCreateChainContext` 会增加系统冗余代码量。

可以考虑将这两个基础类抽象出来，作为基础组件库中的通用组件，供所有系统下的业务使用，从而避免代码冗余。

如果想了解如何实现这一操作，请跟随马哥的思路继续往下阅读。

1. 抽象基础类

定义抽象责任链处理接口，等同于 `OrderCreateChainHandler`。

```
public interface AbstractChainHandler<T> extends Ordered {

    /**
     * 执行责任链逻辑
     *
     * @param requestParam 责任链执行入参
     */
    void handler(T requestParam);

    /**
     * @return 责任链组件标识
     */
    String mark();
}
```

`mark` 方法是做什么的？接口增加 `mark` 方法，以便不同业务使用不同的标识。

假设项目中有两个业务场景：订单下单和用户创建都需要责任链模式去验证，`mark` 就是用来进行分组，在业务中进行调用责任链时传递不同的 `mark` 方法参数，通过该参数找到对应的一组责任链具体实现类集合。

定义抽象责任链上下文，等同于 `OrderCreateChainContext`。可以看到保存责任链处理类的容器从 `List` 改为了 `Map`，这样可以方便扩展更多的不同业务责任链子类。

通过实现 `CommandLineRunner` 接口在 `SpringBoot` 启动完成后，执行钩子函数将所有实现责任链抽象接口的实现类进行注册到责任链上下文中。

```
public final class AbstractChainContext<T> implements CommandLineRunner {

    private final Map<String, List<AbstractChainHandler>> abstractChainHandlerContainer =
        Maps.newHashMap();

    /**
     * 责任链组件执行
     *
     */
}
```

```

    * @param requestParam 请求参数
    */
    public void handler(String mark, T requestParam) {
        abstractChainHandlerContainer.get(mark).stream()
            .sorted(Comparator.comparing(Ordered::getOrder)).forEach(each ->
each.handler(requestParam));
    }

    @Override
    public void run(String... args) throws Exception {
        // 获取 Spring IOC 容器中所有 AbstractChainHandler 接口实现
        Map<String, AbstractChainHandler> chainFilterMap =
ApplicationContextHolder.getBeansOfType(AbstractChainHandler.class);
        chainFilterMap.forEach((beanName, bean) -> {
            List<AbstractChainHandler> abstractChainHandlers =
abstractChainHandlerContainer.get(bean.mark());
            if (abstractChainHandlers == null) {
                abstractChainHandlers = new ArrayList();
            }
            abstractChainHandlers.add(bean);
            // 根据 mark 标识将责任链模式分类，放入责任链容器上下文中
            abstractChainHandlerContainer.put(bean.mark(), abstractChainHandlers);
        });
    }
}

```

这两个接口已经被放置在基础组件库中，如果业务需要使用责任链模式，则无需重新定义。

现在，让我们来看看业务代码需要编写哪些逻辑。

2. 抽象业务接口

让我们先进行头脑风暴，思考一下这个接口的用途是什么？

```

// 订单创建责任链过滤器
public interface OrderCreateChainFilter<T extends OrderCreateCommand> extends
AbstractChainHandler<OrderCreateCommand> {

    @Override
    default String mark() {

```

```
        return OrderChainMarkEnum.ORDER_CREATE_FILTER.name();
    }
}
```

首先，如果没有 `OrderCreateChainFilter` 接口，会是什么样的场景？

- 由于责任链处理子类都需要依赖顶级抽象接口，因此要想知道某个业务场景下有多少具体子类是相当困难的。
- 由于责任链处理子类都需要实现 `Mark` 方法，实际上某一类责任链子类的 `Mark` 方法返回值是相同的。

通过在业务层面上抽象出一个具体业务责任链接口，就能很好地解决上述两个问题。

现在，让我们继续探讨责任链子类的编写。实际上，改动并不多，只需要将之前的 `OrderCreateChainHandler` 实现接口改为 `OrderCreateChainFilter` 即可。

```
// 订单创建参数必填检验
@Component
public final class OrderCreateParamNotNullChainHandler implements
OrderCreateChainFilter<OrderCreateCommand> {

    @Override
    public void handler(OrderCreateCommand requestParam) {
        // 逻辑执行
    }

    @Override
    public int getOrder() {
        return 0;
    }
}

// 订单创建参数正确性检验
@Component
public final class OrderCreateParamVerificationChainHandler implements
OrderCreateChainFilter<OrderCreateCommand> {

    @Override
    public void handler(OrderCreateCommand requestParam) {
        // 逻辑执行
    }
}
```



```

    }

    @Override
    public int getOrder() {
        return 1;
    }
}

// 订单创建商品 SKU 库存验证
@Component
public final class OrderCreateProductSkuStockChainHandler implements
OrderCreateChainFilter<OrderCreateCommand> {

    @Override
    public void handler(OrderCreateCommand requestParam) {
        // 逻辑执行
    }

    @Override
    public int getOrder() {
        return 2;
    }
}

```

3. 业务使用

在具体业务场景中使用时，与之前相比并没有太大的差别。除了增加了 Mark 标识外，没有进行其他变更。

```

@RequiredArgsConstructor
public class OrderServiceImpl implements OrderService {

    private final AbstractChainContext<OrderCreateCommand> abstractChainContext;

    public String createOrder(OrderCreateCommand requestParam) {
        // 责任链模式：执行订单创建参数验证
        abstractChainContext.handler(OrderChainMarkEnum.ORDER_CREATE_FILTER.name(),
requestParam);
    }
}

```

```
}  
}
```

文末总结

本文详细介绍了责任链模式的概念，并通过电商下单场景模拟了真实使用场景。

为了复用责任链接口定义和上下文，我们通过抽象的方式将责任链门面接口加入到基础组件库中，实现快速接入责任链的目的。

虽然在本文中，我们没有使用 `boolean` 类型的返回值，而是通过异常来终止流程，但在后续的增强中，我们可以考虑添加布尔类型的返回值。

此外，我们还可以在 `AbstractChainHandler` 中增加是否异步执行的方法，以提高方法执行性能和减少接口响应时间。

架构设计总是在不断演进，本文的设计也有优化和进步的空间，让我们继续探索责任链模式的更多可能性。

更新: 2023-07-29 17:49:00

原文: <<https://www.yuque.com/magestack/12306/oly3msky0plpogzt>>